

University of Engineering and Technology

Compiler Construction Lab Mini Compiler Project Report

Submitted By:

Member 1: Munees Tariq (2023-CS-32)

Member 2: Muhammad Mahad Sheraz (2023-CS-36)

Section: A

Supervisor:

Mam Aliya Farooq

Spring 2026

Contents

1	Introduction	2
2	Grammar Specification	2
2.1	Original and LL(1) Grammar	2
3	Project Architecture and Source Files	3
3.1	Source Code Files	3
3.2	How the Compiler Was Built	4
4	Module Descriptions	4
4.1	Lexical Analyzer (Lab 3)	4
4.2	Recursive Descent Parser (Lab 4)	4
4.3	Non-Recursive Predictive Parser (Lab 5)	4
4.4	LR Parser (New Module)	5
4.5	Symbol Table Manager (Lab 7, Extended)	5
4.6	Error Handler (Bonus Task)	5
5	Construction of the LR Parsing Table	5
6	Module Integration	5
7	Sample Input and Output	5
7.1	Sample Input	5
7.2	Lexer Output	6
7.3	LR Parser Trace	6
7.4	Symbol Table Dump	6
7.5	Error Handler Output	6
8	Limitations	7
9	References	7

1 Introduction

This report details the design and implementation of the mini compiler for the CS-471L Compiler Construction Lab final project. The compiler is built for a subset of the MicroJava programming language. It integrates all the lab modules developed during the semester, along with additional components such as an LR Parser, Symbol Table Manager, and Error Handling.

2 Grammar Specification

The language subset used is MicroJava. The grammar supports class declarations, variable declarations, methods, statements (if, while, return, print), expressions, and assignments. The original grammar, as well as the left-factored and non-left-recursive version used for top-down parsing, are specified in the accompanying documentation.

2.1 Original and LL(1) Grammar

The full grammar in BNF form can be found in docs/grammar.md.

```
# Section 1 Grammar

This project uses a compact MicroJava subset with classes, fields, methods, local declarations,
assignments, calls, loops, conditionals, returns, printing, arrays, arithmetic, relational
expressions, and character constants.

## Original BNF

The original grammar keeps expression precedence left-recursive for the LR parser.

- 'Program ::= class id { MemberList }'
- 'MemberList ::= Member MemberList | epsilon'
- 'Member ::= Type id MemberAfter'
- 'MemberAfter ::= ; | [ number ] ; | ( ParamListOpt ) Block'
- 'Type ::= int | char | boolean | void'
- 'ParamListOpt ::= ParamList | epsilon'
- 'ParamList ::= Type id ParamListTail'
- 'ParamListTail ::= , Type id ParamListTail | epsilon'
- 'Block ::= { StmtList }'
- 'StmtList ::= Stmt StmtList | epsilon'
- 'Stmt ::= Type id VarDeclAfter ; | id StmtIdTail ; | return ExprOpt ; | if ( Expr ) Stmt | while ( Expr
    ) Stmt | Block ; | print ( Expr ) ;'
- 'VarDeclAfter ::= [ number ] | epsilon'
- 'StmtIdTail ::= AssignTarget = Expr | ( ArgListOpt )'
- 'AssignTarget ::= [ Expr ] | epsilon'
- 'ExprOpt ::= Expr | epsilon'
- 'ArgListOpt ::= ArgList | epsilon'
- 'ArgList ::= Expr ArgListTail'
- 'ArgListTail ::= , Expr ArgListTail | epsilon'
- 'RelOp ::= == | != | < | <= | > | >='
- 'AddOp ::= + | -'
- 'MulOp ::= * | / | %'
- 'Factor ::= id FactorAfter | number | char_const | true | false | ( Expr )'
- 'FactorAfter ::= ( ArgListOpt ) | [ Expr ] | epsilon'
- 'Expr ::= Expr RelOp AddExpr | AddExpr'
- 'AddExpr ::= AddExpr AddOp Term | Term'
- 'Term ::= Term MulOp Factor | Factor'

## LL(1) Transformed Grammar

- 'Program ::= class id { MemberList }'
- 'MemberList ::= Member MemberList | epsilon'
- 'Member ::= Type id MemberAfter'
- 'MemberAfter ::= ; | [ number ] ; | ( ParamListOpt ) Block'
- 'Type ::= int | char | boolean | void'
```

```

- 'ParamListOpt ::= ParamList | epsilon'
- 'ParamList ::= Type id ParamListTail'
- 'ParamListTail ::= , Type id ParamListTail | epsilon'
- 'Block ::= { StmtList }'
- 'StmtList ::= Stmt StmtList | epsilon'
- 'Stmt ::= Type id VarDeclAfter ; | id StmtIdTail ; | return ExprOpt ; | if ( Expr ) Stmt | while ( Expr
    ) Stmt | Block ; | print ( Expr ) ;'
- 'VarDeclAfter ::= [ number ] | epsilon'
- 'StmtIdTail ::= AssignTarget = Expr | ( ArgListOpt )'
- 'AssignTarget ::= [ Expr ] | epsilon'
- 'ExprOpt ::= Expr | epsilon'
- 'ArgListOpt ::= ArgList | epsilon'
- 'ArgList ::= Expr ArgListTail'
- 'ArgListTail ::= , Expr ArgListTail | epsilon'
- 'Expr ::= RelExpr'
- 'RelExpr ::= AddExpr RelExprTail'
- 'RelExprTail ::= RelOp AddExpr | epsilon'
- 'RelOp ::= == | != | < | <= | > | >='
- 'AddExpr ::= Term AddExprTail'
- 'AddExprTail ::= AddOp Term AddExprTail | epsilon'
- 'AddOp ::= + | -'
- 'Term ::= Factor TermTail'
- 'TermTail ::= MulOp Factor TermTail | epsilon'
- 'MulOp ::= * | / | %'
- 'Factor ::= id FactorAfter | number | char_const | true | false | ( Expr )'
- 'FactorAfter ::= ( ArgListOpt ) | [ Expr ] | epsilon'

Transformation notes:
- Direct left recursion in expressions was replaced by 'AddExprTail', 'TermTail', and related tail non-
  terminals.
- Common prefixes such as member declarations and statement forms were left-factored.
- Empty productions are written as 'epsilon'.

```

3 Project Architecture and Source Files

Our project follows a modular architecture. All source code is organized within the `src/` directory. Each module corresponds to a specific compilation phase or support structure. The main integration point ties these modules together.

3.1 Source Code Files

Below is a detailed breakdown of what each file in the `src/` directory does:

- **compiler.c:** This is the main entry point (dispatcher) of the integrated compiler. It contains the `main` function that processes command-line arguments to determine which phase to run (lexer, recursive, predictive, lr, or all). It ties all other modules together.
- **lexer.c & lexer.h:** Implement the lexical analyzer (Lab 3). It reads the source program using double buffering and produces a token stream. It tracks line and column numbers.
- **recursive_descent_parser.c:** Implements the top-down recursive descent parser (Lab 4). Contains mutually recursive functions for each grammar non-terminal.
- **predictive_parser.c:** Implements the non-recursive LL(1) predictive parser (Lab 5). It uses an explicit stack and an LL(1) parsing table to parse tokens without recursion.

- **lr_parser.c:** Implements the SLR(1) bottom-up parser (New Module). It uses shift and reduce operations based on an action and goto table.
- **lr_tables.h:** A header file containing the hardcoded SLR(1) action and goto tables used by the LR parser.
- **symbol_table.c & symbol_table.h:** Implement the Symbol Table Manager (Lab 7 extended). A hash-based structure that stores identifier attributes and handles nested scopes.
- **error_handler.c & error_handler.h:** Implement the Error Handler (Bonus task). Centralizes error reporting and provides panic-mode recovery functionality for parsers.
- **grammar_tables.py:** A utility Python script used offline to automatically generate the FIRST/FOLLOW sets, LL(1) parsing table, and LR parsing tables.

3.2 How the Compiler Was Built

The project includes a robust build system to compile the source code on both Linux and Windows environments.

- **Linux/Unix (Makefile):** Running `make` compiles individual object files into an `obj/` directory and links them to create the integrated `compiler` executable. It also provides targets like `make lexer`, `make parse`, and `make clean`. The Makefile handles conditional compilation using preprocessor macros (e.g., `-Dmain=lexer_main`) to allow individual modules to be built standalone or integrated.
- **Windows (build.bat):** A batch script that invokes the `gcc` compiler to build the standalone executables (`lexer.exe`, `recursive.exe`, `predictive.exe`, `lr.exe`) and the final integrated `compiler.exe`.

4 Module Descriptions

4.1 Lexical Analyzer (Lab 3)

The lexical analyzer (`src/lexer.c`) tokenizes the input source program. It groups characters into tokens (keywords, identifiers, numbers, operators) while ignoring whitespace and comments. It tracks the line and column number for error reporting.

4.2 Recursive Descent Parser (Lab 4)

The recursive descent parser (`src/recursive_descent_parser.c`) implements mutually recursive functions for each non-terminal. It uses a left-factored and non-left-recursive version of the grammar.

4.3 Non-Recursive Predictive Parser (Lab 5)

The predictive parser (`src/predictive_parser.c`) is an LL(1) parser avoiding recursion via an explicit stack. The FIRST and FOLLOW sets used for the LL(1) parsing table are in `docs/first_follow.md`, and the table is in `docs/ll1_table.md`.

4.4 LR Parser (New Module)

The LR parser (`src/lr_parser.c`) is a bottom-up SLR(1) parser. It parses the original grammar by performing shift and reduce operations using the Action and Goto tables.

4.5 Symbol Table Manager (Lab 7, Extended)

The symbol table (`src/symbol_table.c`) uses a hash-based structure for fast lookups and a linked scope chain for nested scopes. It stores identifier name, kind, type, scope level, and source line number.

4.6 Error Handler (Bonus Task)

The error handler (`src/error_handler.c`) manages lexical, syntactic, and semantic errors with detailed line and column numbers. It supports panic-mode recovery.

5 Construction of the LR Parsing Table

The SLR(1) parsing technique was chosen for the LR parser. The steps to construct the SLR(1) parsing table are as follows:

1. **Augment the Grammar:** A new start symbol is added.
2. **LR(0) Item Sets:** Construct the canonical collection of LR(0) items.
3. **Goto Graph:** Transitions defined on terminals and non-terminals.
4. **Action Table:** Shift actions for terminal transitions. Reduce actions for completed items for lookahead symbols in the FOLLOW set. Accept action for the augmented start production.
5. **Goto Table:** Non-terminal transitions are added.

The full LR Action and Goto tables are available in `docs/lr_table.md`.

6 Module Integration

All modules are integrated via `src/compiler.c`. The command-line interface acts as the main dispatcher. The lexer provides a token stream that any parser can consume. The parsers communicate with the Symbol Table Manager for semantic checks and the Error Handler to report and recover from issues gracefully.

7 Sample Input and Output

7.1 Sample Input

A valid MicroJava program, `hello.mj`:

```

class Hello {
    void main() {
        print(1 + 2 * 3);
        return;
    }
}

```

7.2 Lexer Output

Sample tokens extracted (for full output, see `output/lexer_output.txt`):

Position	Token Type	Lexeme
1:1	KEYWORD	'class'
1:7	IDENTIFIER	'Hello'
1:13	PUNCTUATION	'{'
...	...	...
7:1	EOF	''

7.3 LR Parser Trace

A snippet of the trace showing stack contents, remaining input, and actions (for full output, see `output/lr_parser_output.txt`):

Step	Stack	Symbols	Input	Action
0	0		class Hello	s6
...	...	...	...	...
Accept.				

7.4 Symbol Table Dump

When semantic analysis is enabled:

Name	Kind	Type	Scope	Line
main	function	void	0	2
Hello	class	class	0	1

7.5 Error Handler Output

When errors are encountered (panic-mode recovery sample):

```

[Error Handler Output]
Error at line 3, column 12: Missing semicolon
Recovering... Skipped 3 tokens.

```

8 Limitations

- The compiler is a front-end only; it does not generate intermediate representation or target machine code.
- Semantic checking ensures proper scope rules but does not implement comprehensive type checking for complex expressions.
- Error recovery uses basic panic-mode techniques (skipping tokens until a synchronization point).

9 References

1. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Pearson.
2. Course Materials and Lab Manuals provided by Instructor Aliya Farooq.